Descripción integral de la aplicación FlowTrim: propósito, funcionalidades, módulos, tecnología, arquitectura, reglas de negocio, base de datos, estándares, pruebas, ejecución local y estado de entrega.

Estado: MVP completo con las iteraciones visuales, de localización y de calidad interactiva EP-01, EP-02 y EP-03 entregadas.

Preparado para Ander
Octubre de 2026

1. Resumen ejecutivo

Qué existe actualmente y qué se ha entregado.

FlowTrim es una aplicación full-stack de finanzas personales enfocada en suscripciones digitales recurrentes. Ayuda a centralizar suscripciones, entender su costo mensual equivalente, anticipar renovaciones, comparar escenarios de cancelación e identificar servicios que podrían estar infrautilizados.

El producto es determinista y transparente por diseño. No se conecta a bancos, no confirma transacciones reales, no verifica la actividad del usuario y no cancela servicios de terceros. El usuario introduce sus datos y FlowTrim explica cómo se produce cada número.

El MVP funcional está completo. Las iteraciones posteriores agregaron identidad visual, landing page, workspace responsive, temas claro/oscuro, localización inglés/español, motion, navegación fija, scroll aislado de tablas, modales CRUD, iconos y protección de formularios con cambios sin guardar.

Área	Estado actual
Backend	API ASP.NET Core .NET 10, persistencia PostgreSQL y 90 pruebas backend aprobadas
Frontend	SPA Angular 21 standalone, 16 pruebas frontend aprobadas y build de producción exitoso
Base de datos	PostgreSQL 17 mediante Docker Compose y migraciones EF Core aplicadas explícitamente
UX	Landing, workspace, temas, localización, CRUD modal y responsive entregados
Entrega	Frontend main y tracker main incluyen EP-03; backend main incluye HU-06

2. Qué es FlowTrim

Introducción al producto y valor para el usuario.

FlowTrim nace de un problema financiero cotidiano: las suscripciones son fáciles de contratar y fáciles de olvidar. Un cobro mensual puede parecer pequeño de forma aislada, pero una cartera de servicios crea un gasto recurrente constante que se vuelve difícil de interpretar cuando los ciclos, las monedas y las fechas de renovación están distribuidos entre distintos proveedores.

FlowTrim convierte ese gasto invisible en una superficie de control explícita. El usuario puede registrar una suscripción, ver su costo normalizado, entender qué está próximo a cobrarse, registrar un uso declarado y simular qué ocurriría al recortar determinados servicios.

Objetivo del producto

- Ofrecer un único lugar para consultar suscripciones, importes y próximas fechas de cobro.
- Mostrar el costo mensual equivalente y la proyección anual realista.
- Prevenir renovaciones inesperadas con cuentas regresivas y alertas visuales.
- Revelar servicios potencialmente infrautilizados mediante costo por uso declarado.
- Comparar escenarios de cancelación sin persistir una cancelación real.
- Demostrar una aplicación full-stack completa, testeable y reproducible.

Principios del producto

- Transparencia: cada número mostrado debe tener una fórmula u origen visible.
- Accionabilidad: comparar escenarios en lugar de limitarse a listar gastos.
- Honestidad: separar cobros esperados de transacciones confirmadas y uso declarado de uso verificado.

- Simplicidad técnica: mantener un backend desplegable y evitar servicios especulativos.
- Calidad demostrable: pruebas, validaciones, errores controlados y ejecución reproducible.

3. Funcionalidades principales

Qué puede hacer un usuario actualmente.

Funcionalidad	Descripción
Gestión de suscripciones	Crear, listar, buscar, editar y eliminar suscripciones.
Resumen financiero	Consultar equivalente mensual, proyección anual y servicios activos por moneda.
Próximos cobros	Revisar cobros vencidos, de hoy, de 3 días, de 7 días y posteriores.
Uso declarado	Registrar cantidades de uso por fecha y notas opcionales.
Costo por uso	Calcular costo por uso declarado sin dividir por cero.
Simulador	Comparar gasto actual y escenario de recorte a 1, 12 y 24 meses.
Landing y workspace	Entender el producto antes de entrar al centro de control financiero.
Temas e idiomas	Usar modo claro/oscuro e interfaz persistente en inglés/español.
CRUD modal	Gestionar suscripciones con modales accesibles e iconos reutilizables.

4. Roadmap entregado

Épicas e historias completadas.

ID	Alcance	Estado
EP-00	Preparación de repositorio, entorno y arquitectura	Completada
HU-01	Registrar suscripción	Completada
HU-02	Consultar, editar y eliminar suscripciones	Completada
HU-03	Resumen financiero	Completada
HU-04	Próximos cobros y cuenta regresiva	Completada
HU-05	Registros de uso declarado	Completada
HU-06	Costo por uso	Completada
HU-07	Simulador de cancelación	Completada
HU-08	Integración Angular con API	Completada
HU-09	Validación y errores	Completada
HU-10	Pruebas de reglas financieras	Completada
HU-11	Documentación y demo	Completada
EP-01	Rediseño visual / HU-16 a HU-24	Completada
EP-02	Localización y motion / HU-25 a HU-27	Completada
EP-03	Interacción y calidad responsive / HU-28 a HU-32	Completada

Las historias posteriores al MVP permanecen planificadas: HU-12 autenticación y aislamiento multiusuario, HU-13 notificaciones externas, HU-14 importación CSV y HU-15 extracción analítica condicional a Python/FastAPI.

5. Arquitectura del sistema

Cómo está organizado FlowTrim.

FlowTrim es un monolito modular con backend .NET por capas y frontend Angular independiente. El backend es un único servicio desplegable. Las reglas de dominio permanecen separadas de la infraestructura y la API expone DTOs, no entidades EF Core.

```
Navegador
-> Componentes Angular standalone
-> Servicios HTTP tipados
-> Endpoints Minimal API ASP.NET Core
-> Handlers de Application
-> Repositorios de Infrastructure
-> EF Core / Npgsql
-> PostgreSQL
```

Regla de dependencias del backend

Capa	Responsabilidad	Dependencias permitidas
FlowTrim.Api	Endpoints HTTP, DI, CORS, configuración y errores	Application, Infrastructure
FlowTrim.Application	Casos de uso, DTOs, contratos y orquestación	Domain
FlowTrim.Domain	Entidades, reglas y calculadoras financieras puras	Ninguna
FlowTrim.Infrastructure	EF Core, PostgreSQL, migraciones, repositorios y health checks	Application, Domain
FlowTrim.Tests	Pruebas xUnit de dominio y aplicación	Proyectos bajo prueba

Estructura frontend

- Componentes standalone con TypeScript estricto y templates estrictos.
- Features para subscriptions, summary, simulator y landing.
- Servicios core para errores HTTP, temas e internacionalización.
- SCSS y custom properties CSS para Carbon Signal y sus temas.
- Iconos SVG y modales nativos reutilizables sin dependencias innecesarias.
- Signals para estado local, sin un segundo state manager global.

6. Modelo de dominio y reglas de negocio

El motor financiero determinista.

Subscription

Subscription contiene identificador, nombre, categoría opcional, importe, moneda, ciclo de cobro, próxima fecha, estado y timestamps. Su factory protege las invariantes y no permite setters públicos que las eviten.

UsageRecord

UsageRecord pertenece a una suscripción y almacena fecha, cantidad de uso no negativa y notas opcionales. El consumo es declarado por el usuario y no se verifica de forma independiente.

Fórmulas financieras

Indicador	Regla	Caveat
Equivalente mensual	Importe mensual; importe anual dividido entre 12	Los anuales se redondean a dos decimales con AwayFromZero.
Proyección anual	Suma de equivalentes activos multiplicada por 12	Se calcula por moneda y nunca se convierten monedas.
Ahorro potencial	Equivalente mensual seleccionado multiplicado por horizonte	Escenario; excluye impuestos, penalizaciones y cambios de precio.
Días hasta cobro	NextBillingDate menos fecha de referencia del servidor	Las fechas pasadas se clasifican como vencidas.
Costo por uso	Costo del periodo dividido entre usos declarados	Sin registros o con cero usos: no calculable, nunca infinito.

7. API backend

Rutas, contratos y comportamiento.

Método	Ruta	Propósito
GET	/api/health	Disponibilidad de API y base de datos
GET	/api/subscriptions?page=&pageSize=	Lista paginada
GET	/api/subscriptions/{id}	Detalle
POST	/api/subscriptions	Crear suscripción
PUT	/api/subscriptions/{id}	Actualizar suscripción
DELETE	/api/subscriptions/{id}	Eliminar suscripción
GET	/api/summary	Resumen financiero por moneda
GET	/api/subscriptions/upcoming	Próximos cobros y alertas
POST	/api/subscriptions/{id}/usage-records	Registrar uso declarado
GET	/api/subscriptions/{id}/usage-records	Listar usos
GET	/api/subscriptions/{id}/cost-per-use	Costo por uso declarado
POST	/api/simulations	Simulación no persistente

Estándares HTTP y errores

- 201 Created para creación exitosa de suscripciones y usos.
- 204 No Content para eliminación exitosa.
- 400 ValidationProblemDetails para entradas inválidas.
- 404 ProblemDetails para recursos inexistentes.
- 500 genérico sin stack traces, secretos ni detalles internos.
- CORS configurable y restringido al origen de desarrollo por defecto.

8. Base de datos y persistencia

Cómo se crea y evoluciona PostgreSQL.

Docker Compose ejecuta PostgreSQL 17. La imagen oficial crea la base y el usuario iniciales a partir de POSTGRES_USER, POSTGRES_PASSWORD y POSTGRES_DB del archivo local .env. Después, EF Core crea el esquema mediante migraciones aplicadas explícitamente con dotnet ef database update.

```
flowtrim-postgres
PostgreSQL 17
puerto host: 5433 en el entorno actual
puerto interno: 5432
base: flowtrim
volumen: flowtrim_pgdata
```

Tablas

Tabla	Propósito	Restricciones
subscriptions	Servicios recurrentes y facturación	amount > 0; checks de ciclo y estado; índice por next_billing_date
usage_records	Historial de uso declarado	usage_count >= 0; foreign key; ON DELETE CASCADE
__EFMigrationsHistory	Migraciones EF aplicadas	Mantenida por EF Core

Los identificadores de base de datos usan snake_case. El dinero usa numeric(18,2), los timestamps timestamptz, las fechas date y los enums texto legible con CHECK constraints.

9. Frontend y sistema visual

La experiencia actual de FlowTrim.

El frontend tiene dos capas visuales. La landing pública introduce el producto, explica el problema y muestra un preview. El workspace /app es el centro operativo con sidebar fijo, temas, selector de idioma, estado de API y navegación por features.

Paleta Carbon Signal

Token	Oscuro	Claro	Significado
Primary	#1FC1CE	#087F89	Acciones, foco y marca
Success	#00C483	#008F61	Ahorro y estados saludables
Warning	#FFB547	#A86400	Próximos cobros y cautela
Danger	#FF6B6B	#C63D47	Eliminación y errores
Background	#070B0D	#F3F7F6	Lienzo de la aplicación

Comportamiento UX entregado

- Tema claro/oscuro persistente con fallback a la preferencia del sistema.
- Idioma inglés/español persistente con fallback seguro a inglés.
- Sidebar fijo en desktop y navegación fija en mobile.
- Feature rail con scroll snap nativo.
- Soporte para reduced motion.
- Modal nativo reusable con X, Escape y cierre por backdrop.
- Protección contra pérdida de datos en formularios dirty.

- Iconos SVG con etiquetas accesibles para CRUD.

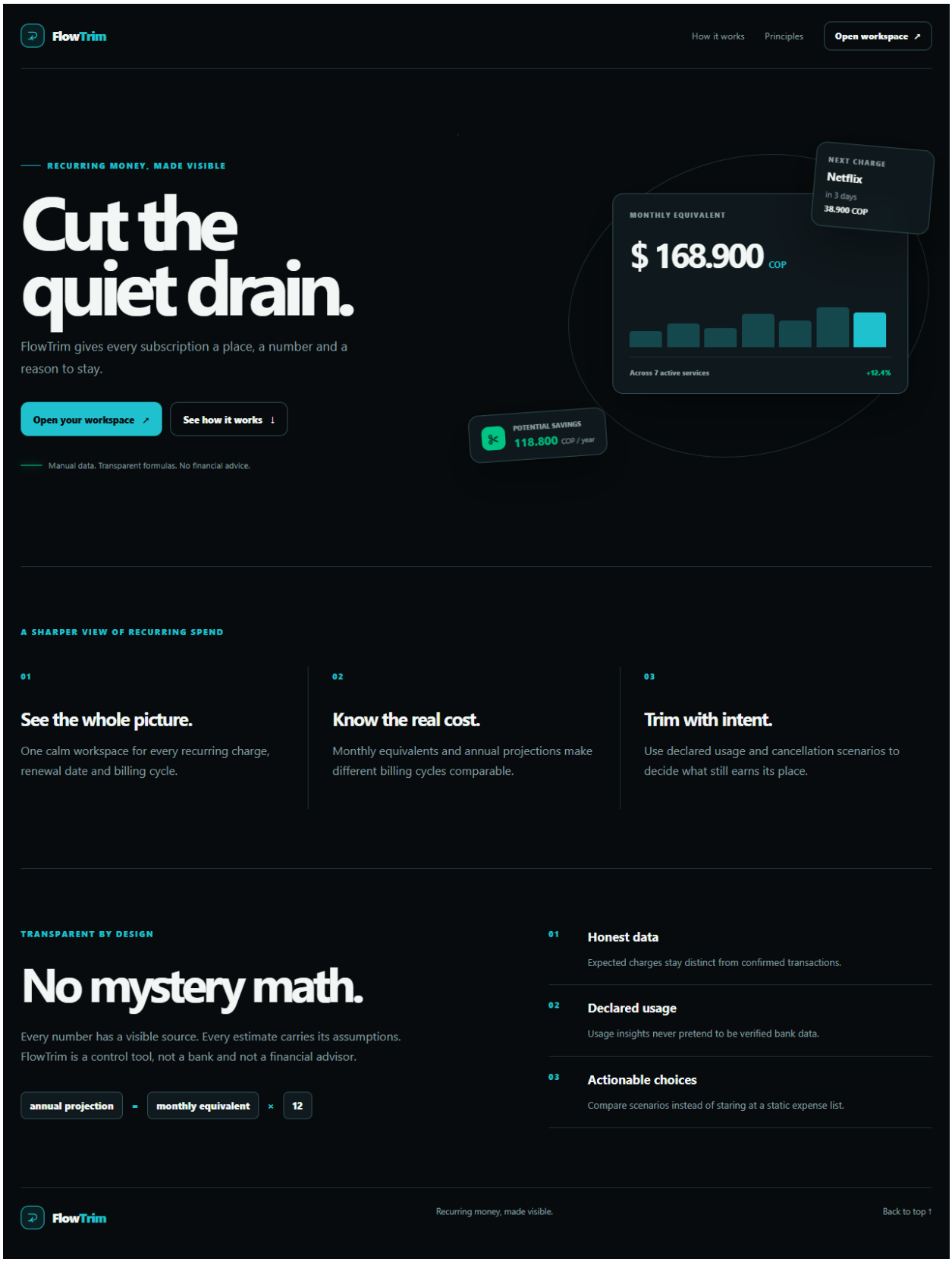


Figura 1. Landing actual con identidad Carbon Signal, preview del workspace y narrativa de producto.

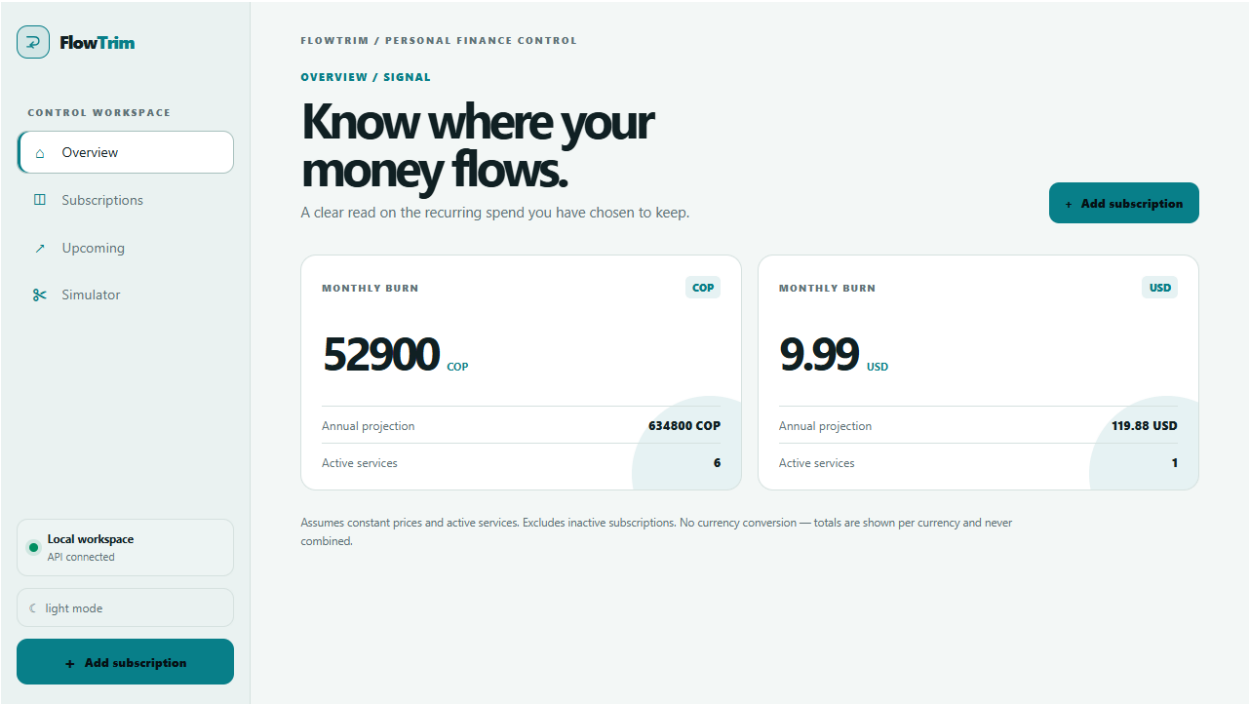


Figura 2. Workspace actual con navegación fija, tema claro y tarjetas financieras por moneda.

10. Ejecución local

Cómo ejecutar el sistema completo en Windows.

Requisitos

- .NET 10 SDK.

- Node.js compatible con Angular 21; el entorno documentado usa Node 22.15.0.

- npm 10.

- Docker Desktop y Docker Compose.

Backend y base de datos

```
cd C:\PROJECTS\PROJECTS\FlowTrim\FlowTrim-backend

# Crear .env desde la plantilla si hace falta
Copy-Item .env.example .env

# Iniciar PostgreSQL
docker compose up -d
docker compose ps

# Aplicar migraciones EF Core
dotnet tool restore
dotnet ef database update --project src/FlowTrim.Infrastructure --startup-project src/FlowTrim.Api

# Configurar User Secrets una vez
dotnet user-secrets set "ConnectionStrings:FlowTrimDb"
"Host=localhost;Port=5433;Database=flowtrim;Username=flowtrim;Password=TU_PASSWORD" --project
src/FlowTrim.Api

# Ejecutar API
dotnet run --project src/FlowTrim.Api --launch-profile http
```

La API queda en http://localhost:5214. El health endpoint debe devolver Healthy para API y base de datos.

Frontend

```
cd C:\PROJECTS\PROJECTS\FlowTrim\FlowTrim-frontend
npm install
npm start
```

El frontend queda en http://localhost:4200. Usa / para la landing y /app para el workspace.

Comandos útiles

```
dotnet build FlowTrim.slnx
dotnet test FlowTrim.slnx
npm test -- --watch=false
npm run build
docker compose down
```

11. Inspección de base de datos con DBeaver

Valores de conexión del entorno local actual.

Campo	Valor
Motor	PostgreSQL 17
Host	localhost
Puerto	5433 en el entorno actual
Base	flowtrim
Usuario	Valor de POSTGRES_USER en .env
Password	Valor de POSTGRES_PASSWORD en .env
Schema	public

En DBeaver abre flowtrim -> Schemas -> public -> Tables. Las tablas principales son subscriptions, usage_records y __EFMigrationsHistory.

```
SELECT * FROM public.subscriptions ORDER BY next_billing_date;

SELECT * FROM public.usage_records ORDER BY usage_date DESC;

SELECT s.name, u.usage_date, u.usage_count, u.notes
FROM public.usage_records AS u
JOIN public.subscriptions AS s ON s.id = u.subscription_id
ORDER BY u.usage_date DESC;
```

DBeaver permite revisar columnas, constraints, índices y foreign keys. La API no necesita estar ejecutándose para conectar; el contenedor PostgreSQL debe estar healthy.

12. Estándares y principios de ingeniería

Convenciones aplicadas en el proyecto.

- Identificadores fuente, rutas API, nombres de base de datos, archivos y elementos técnicos en inglés.

- Comentarios de código en español según la convención existente; la interfaz soporta inglés y español.

- Commits en inglés con Conventional Commits y ramas por tarea.

- Specs, issues nuevas y PRs en inglés para el portfolio público.

- Entidades de dominio protegen invariantes mediante factories y mutación controlada.

- Dinero con decimal; fórmulas puras y testeadas.

- I/O asíncrono con CancellationToken en backend.

- La API nunca expone entidades EF Core directamente.
- Validación en límites de API y dominio.
- RFC 7807 ProblemDetails sin filtraciones internas.
- TypeScript estricto, templates Angular estrictos y componentes standalone.
- Preferencias de usuario en cliente, sin estado adicional en backend.
- Accesibilidad con labels semánticos, focus, teclado y reduced motion.
- Arquitectura preservada: sin microservicios, state manager competidor ni dependencias especulativas.

13. Evidencia de verificación

Pruebas y comprobaciones ejecutadas.

Comprobación	Resultado
Pruebas backend	90 aprobadas
Pruebas frontend después de EP-03	16 aprobadas
Build frontend de producción	Exitoso sin errores ni warnings
Health API real	API y PostgreSQL saludables
Cost-per-use real	200 calculable, 200 no calculable y 404 verificados
Browser QA landing	Cero errores de consola y requests fallidos
Browser QA workspace	Overview con API real, cero errores
Browser QA modal	Modal de creación con labels y orden Cancel/Primary correcto
Browser QA localización	Landing cambió a español y workspace mostró selector

El flujo de verificación es explícito: implementar, compilar, ejecutar unit tests, ejecutar contra PostgreSQL, revisar el UI en un navegador real y documentar el resultado.

14. Seguridad, privacidad y limitaciones

Lo que FlowTrim deliberadamente no afirma.

- Todavía no existe autenticación ni aislamiento por usuario; no debe publicarse con datos privados sin HU-12.
- No existe integración bancaria, detección automática de transacciones ni cancelación de terceros.
- Los registros de uso son declarados y no verificados.
- Las fechas de renovación esperadas no son transacciones bancarias confirmadas.
- No se realiza conversión de monedas.
- No se envían emails, SMS ni notificaciones push.
- El despliegue público sigue diferido porque no se ha elegido proveedor.
- El backend mantiene documentada la advertencia NU1903 de OpenAPI como deuda técnica.
- El frontend mantiene Angular 21 y legacy-peer-deps por la decisión actual de compatibilidad Node/npm.

15. Roadmap pendiente

Trabajo deliberadamente fuera de la entrega actual.

Elemento	Por qué está pendiente
HU-12 autenticación y aislamiento multiusuario	Es necesaria antes de publicar datos privados; no se ha elegido proveedor de identidad.
HU-13 notificaciones externas	Requiere ejecución en background y proveedor email/push.
HU-14 importación CSV	Requiere un modelo futuro de transacciones separado de renovaciones esperadas.
HU-15 analítica Python/FastAPI	No existe una necesidad técnica validada; el motor C# determinista es suficiente.
Despliegue público #27	No se ha seleccionado proveedor ni arquitectura de hosting.
Ciclos adicionales y catálogo de categorías	Decisiones de producto pendientes que no bloquean el MVP.

16. Historial de entrega

Puntos principales fusionados.

Entrega	Resultado
Backend PR #31	HU-01 registrar suscripción
Backend PR #40 / Frontend PR #22	HU-05 uso declarado
Backend PR #41 / Frontend PR #23	HU-06 costo por uso
Frontend PR #42 / Tracker PR #38	EP-01 rediseño visual
Frontend PR #48 / Tracker PR #43	EP-02 localización y motion
Frontend PR #59 / Tracker PR #50	EP-03 interacción y calidad responsive
v0.1.0	Release inicial documentado después del hito MVP P0

Estado actual: backend y frontend main sincronizados. El tracker contiene las specs de EP-01, EP-02 y EP-03. El archivo specs/_TEMPLATE.md ya existía sin seguimiento y se dejó intacto.

17. Evaluación final

Madurez actual del producto.

FlowTrim evolucionó de un MVP funcional a una experiencia de producto coherente. Sus principales fortalezas son la lógica financiera determinista, los supuestos explícitos, la base PostgreSQL reproducible y una identidad frontend cada vez más intencional. El siguiente umbral importante no es otra capa visual: es autenticación y aislamiento de datos antes del despliegue público.

El sistema actual es adecuado para demostraciones locales, portfolio, evaluación técnica y desarrollo continuo. Todavía no debe considerarse un servicio financiero público multiusuario, y la documentación conserva explícitamente esa distinción.